Feature Extraction using an Autoencoder Trained on MNIST

Zayan Afzal
MTH6161 Mini Project

May 2026

1 Introduction

Deep learning models can learn useful patterns directly from data without requiring manually designed features. One common neural network used for this is the autoencoder. An autoencoder compresses an input into a smaller representation and then attempts to reconstruct the original input from this compressed version. Since information must pass through a bottleneck layer, the network is forced to keep only the most important features of the data.

In this project, an autoencoder was trained on the MNIST handwritten digit dataset. The encoder compresses each 28×28 greyscale image into a 2D latent vector, while the decoder reconstructs the image from this compressed representation. A bottleneck dimension of 2 was chosen so the latent space could be visualised directly.

The MNIST dataset contains handwritten digits from 0 to 9. Each image contains

$$28 \times 28 = 784$$

pixel values normalised to the range $[0, 1]$. The project was implemented in Python using PyTorch and trained on Google Colab.

2 Methodology

2.1 Dataset

The MNIST dataset contains 60,000 training images and 10,000 test images. Pixel values were normalised using the PyTorch `ToTensor()` transform. A batch size of 64 was used during training.

2.2 Autoencoder Architecture

The autoencoder consists of an encoder and a decoder. The encoder compresses the image into a 2D latent vector, while the decoder reconstructs the image from this compressed representation. The network architecture used was:

$$784 \to 128 \to 64 \to 2 \to 64 \to 128 \to 784.$$

The hidden layers gradually reduce the amount of information passing through the network before reaching the bottleneck layer. The decoder mirrors this structure to reconstruct the image.

ReLU activation functions were used in the hidden layers. ReLU is defined as

$$f(x) = \max(0, x)$$

and helps avoid vanishing gradients during training. The bottleneck layer used no activation function so that latent values could take any real value. The final decoder layer used a sigmoid activation function to map outputs into the range $[0, 1]$.

2.3 Training Strategy

The Mean Squared Error (MSE) loss function was used:

$$\mathcal{L} = \frac{1}{n} \sum_{i=1}^{n} (x_i - \hat{x}_i)^2,$$

where x_i is the original pixel value and $\hat{x}_i$ is the reconstructed pixel value.
MSE measures the average squared reconstruction error across all pixels. However, MSE also tends to favour smoother outputs, which can contribute to blurry reconstructions. The Adam optimiser was used with a learning rate of 0.001. The model was trained for 12 epochs using shuffled mini-batches of 64 images.

3 Results

3.1 Training Loss

The training loss decreased consistently across all 12 epochs, falling quickly during the first few epochs before gradually slowing as the model approached convergence.

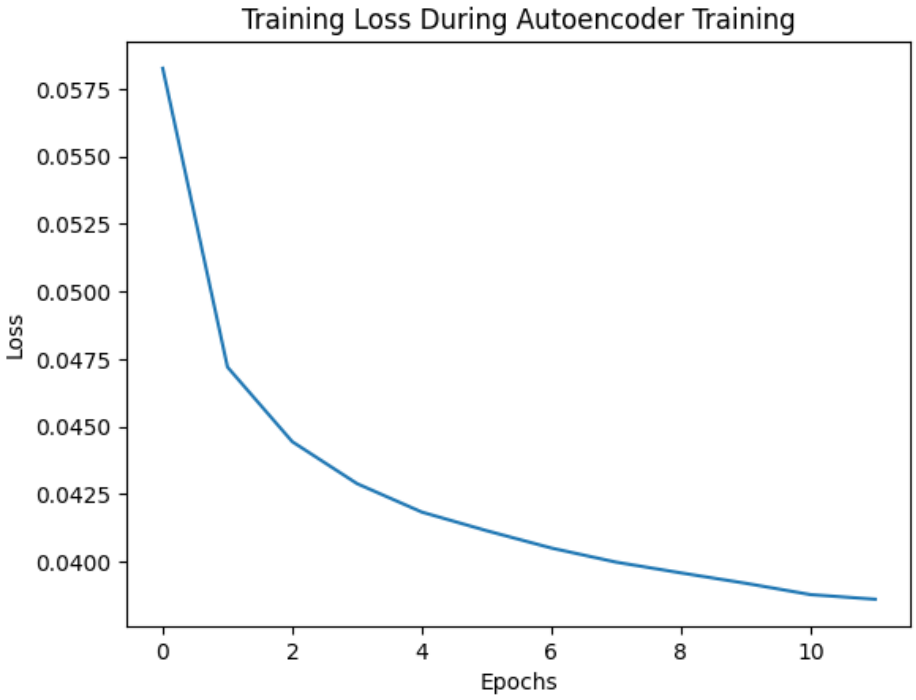

Figure 1: Training loss across 12 epochs.

The smooth decrease in loss suggests that the model trained successfully and that the learning rate was suitable for this task.

3.2 Reconstructed Images

The trained autoencoder was tested on unseen images from the MNIST test set.

Figure 2: Original digits (top row) and reconstructed digits (bottom row).

The reconstructed digits remain recognisable despite the strong compression caused by the bottleneck layer. However, the reconstructions also contain visible blurring and loss of detail. In one example, a 4 was reconstructed as something closer to a 9. This is likely because the two digits have similar shapes, causing their latent representations to overlap when the image is compressed into only two latent variables.

3.3 Latent Space Visualisation

Since the bottleneck has dimension 2, each image can be represented directly as a point in the plane. The latent vectors extracted from the 10,000-image MNIST test set are shown below.

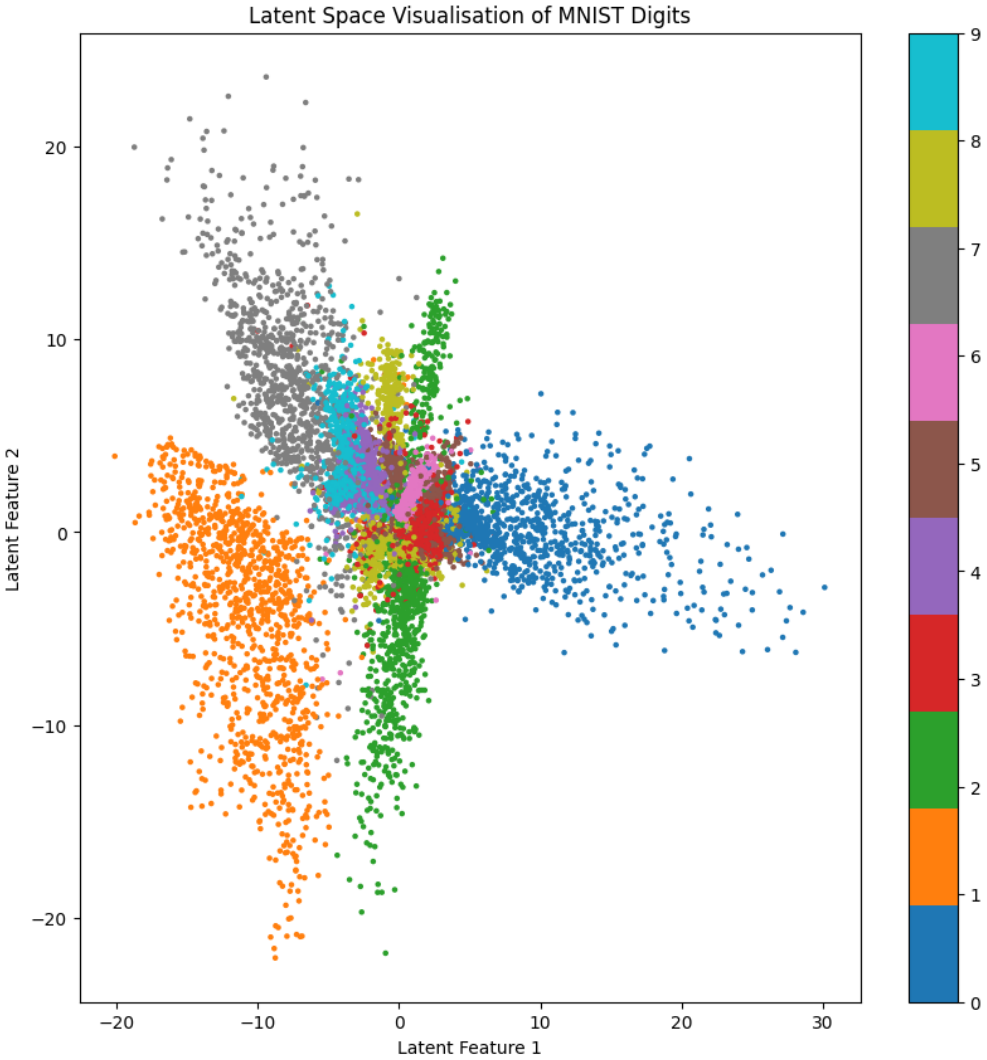

Figure 3: 2D latent space coloured by digit class.

The latent space shows clear clustering behaviour. Digits with distinctive shapes, such as 0 and 1, form clearly separated clusters, while similar digits such as 4 and 9 overlap more strongly.
Digits with related shapes also tend to appear near each other in the latent space. This suggests that the encoder learnt useful compressed features that represent the main differences between the handwritten digits.

The clustering in the latent space suggests that the encoder extracted useful features from the original pixel data despite receiving no label information during training.

3.4 Test Loss

The final Mean Squared Error on the test set was:

$$\mathcal{L}_{\text{test}} \approx 0.0386.$$

Since MSE measures the average squared reconstruction error per pixel, this relatively low value suggests that the decoder reconstructed the overall digit structure reasonably well despite the strong dimensional compression.

Some reconstruction error is unavoidable because compressing 784 values into only two latent variables causes information loss. This can also be seen in the slight blurring present in the reconstructed images.

The relatively low test loss and stable training behaviour suggest that the model generalised reasonably well to unseen data.

4 Discussion

The results showed that the autoencoder could learn meaningful compressed representations of handwritten digit images using only reconstruction as a training signal.

The latent space visualisation was the most interesting result from the model. The encoder reduced each 784 dimensional image into only two latent variables while still preserving visible clustering between digit classes. Digits with distinctive shapes formed clearer clusters, while similar digits showed greater overlap.

The reconstruction results also showed the trade off between compression and reconstruction quality. Because the latent dimension was extremely small, the decoder could only reconstruct approximate versions of the original digits, leading to visible blurring and loss of detail. Increasing the latent dimension would likely improve reconstruction quality by allowing more information to be retained, although this would make direct visualisation of the latent space more difficult.

Another limitation is the use of fully connected layers throughout the network. Fully connected networks do not directly make use of the spatial layout of images. Convolutional layers would likely improve reconstruction quality by detecting local image features more effectively.

5 Conclusion

This project successfully implemented a fully connected autoencoder trained on the MNIST handwritten digit dataset using PyTorch. The model compressed each 28×28 image containing 784 pixel values into only two latent variables before reconstructing the original image.

The model was trained for 12 epochs, which proved sufficient for the training loss to decrease steadily and begin approaching convergence. The loss dropped rapidly during the first few epochs before decreasing more gradually later in training, suggesting that the network first learnt the main structure of the digits before refining smaller details. Training for significantly fewer epochs would likely have caused underfitting, since the loss was still decreasing noticeably during the earlier

stages of training. On the other hand, training for many more epochs would probably only have produced small improvements while increasing training time, as the loss curve had already begun to flatten by the later epochs. The final test loss of approximately 0.0386 suggested that the model generalised reasonably well to unseen test images.

The reconstructed images showed that the decoder preserved the main structure of most digits despite the strong compression caused by the bottleneck layer. However, the reconstructions also contained visible blurring and occasional errors. In particular, one digit 4 was reconstructed more like a 9. This was likely caused by the overlap between visually similar digits in the latent space. Since the encoder compressed 784 pixel values into only two latent variables, some finer structural differences between similar digits were lost during compression.

The latent space visualisation was one of the most important results from this project. Digits with distinctive shapes, such as 0 and 1, formed clearer clusters, while visually similar digits such as 4 and 9 overlapped more strongly. This showed that the encoder successfully extracted meaningful features from the handwritten digits even though the model was trained without using class labels.

Overall, the project demonstrated how an autoencoder can perform unsupervised feature extraction and dimensionality reduction using reconstruction alone. It also highlighted the trade-off between compression and reconstruction quality, where reducing the latent space to only two variables improves visualisation but also increases information loss during reconstruction.

References

[1] M. Hanada, *MTH767/6161 Neural Networks and Deep Learning Lecture Notes*, 2026.

[2] Y. LeCun, C. Cortes, and C. J. C. Burges, *The MNIST Database of Handwritten Digits*. Available at: `http://yann.lecun.com/exdb/mnist/`